

Рассчетная работа 2 сем

Башура А.О., 421702

10 апреля 2025 г.

1 Цель:

- Повторить теорию графов.
- Формализовать алгоритм по шагово, выданный преподавателем

2 Задачи:

- Формализовать алгоритм в редакторе КВЕ на нескольких примерах
- Подготовить отчет о проделанной работе в Latex

3 Вариант:

Для расчетной работы мне был выдан вариант **5.29**. Для хранения графов необходимо использовать списки смежности, а также граф должен быть взвешенным и неориентированным.

4 Теоритические сведения

Некоторые примеры, будет выглядеть формализция графа и других деталей, которые понадобятся для формализации текста

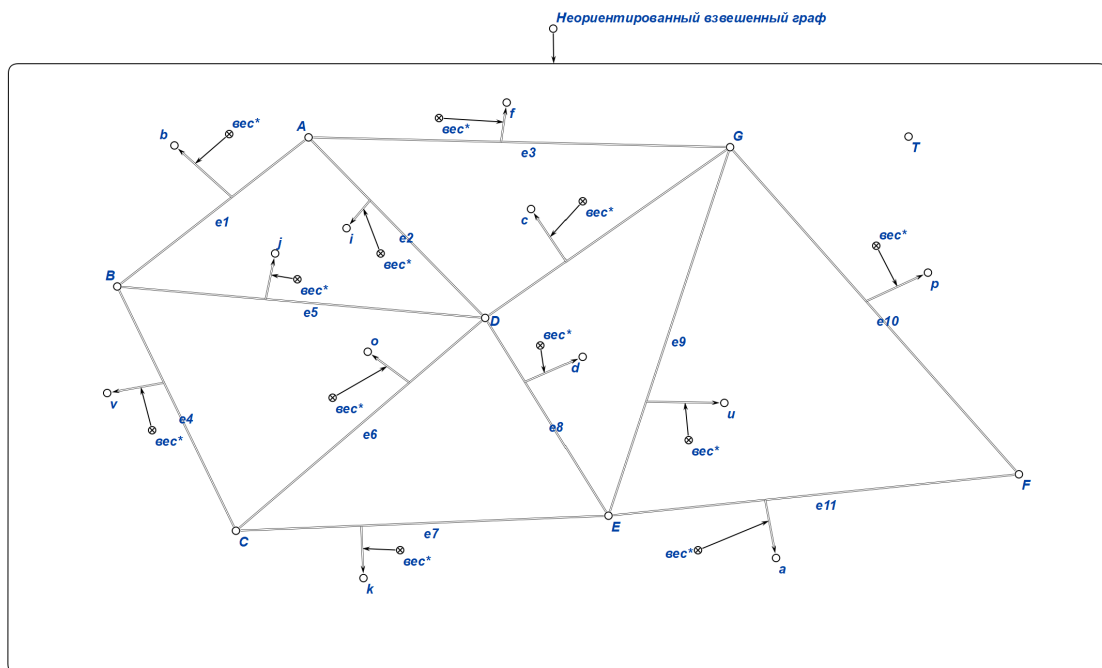


Рис. 1: Формализация графа

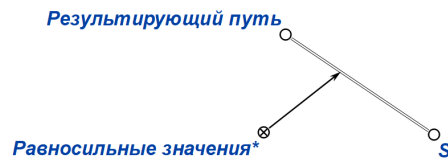


Рис. 2: Представление переменной, которая хранит в себе длину пути между 2 вершинами графа

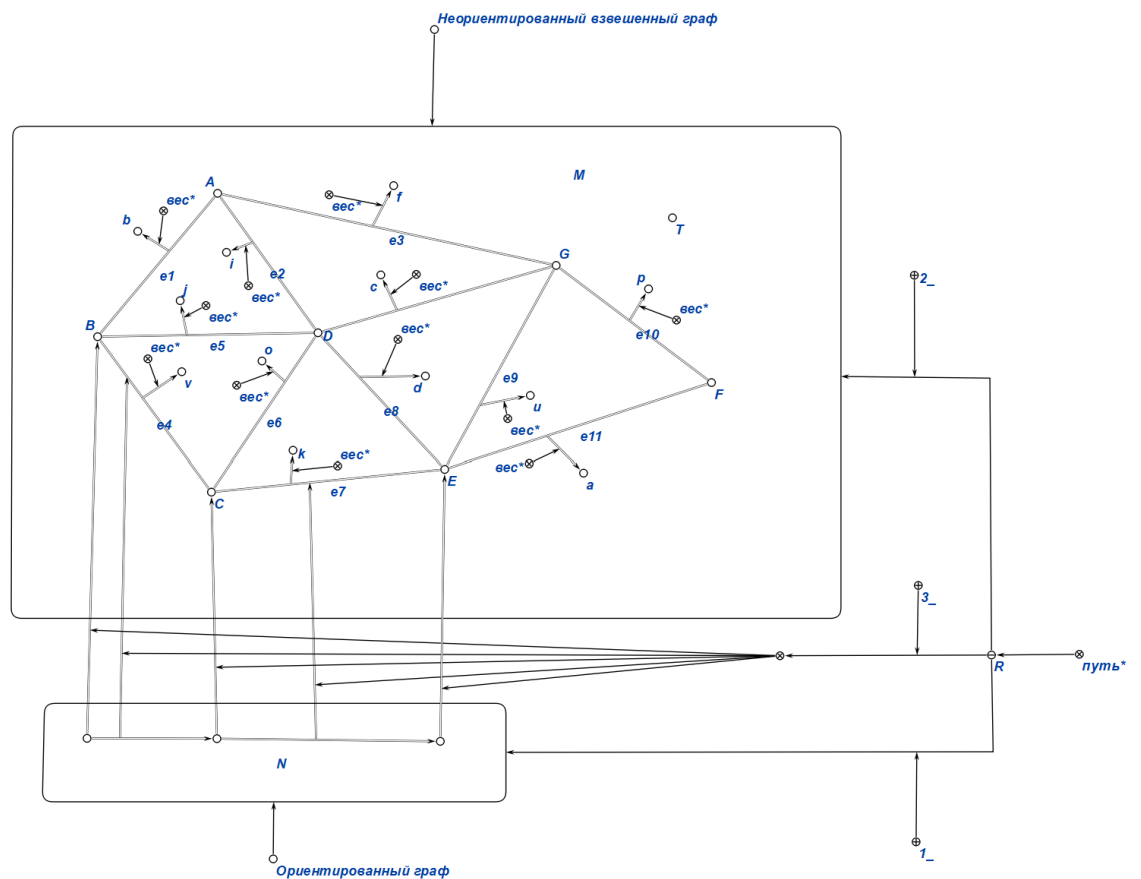


Рис. 3: Формализация кратчайшего пути из В в Е(заранее известно, что это кратчайший путь)

В моем варианте нужно найти кратчайшее расстояние от одной вершины ко всем остальным. На Рис. 3 Я показал только кратчайший путь от стартовой вершины к одной из оставшихся. Аналогично показываются все другие пути, но для упрощения я показал только один.

5 Тестовые примеры

5.1 Тест1

В этом тестовом примере стартовая вершина 1

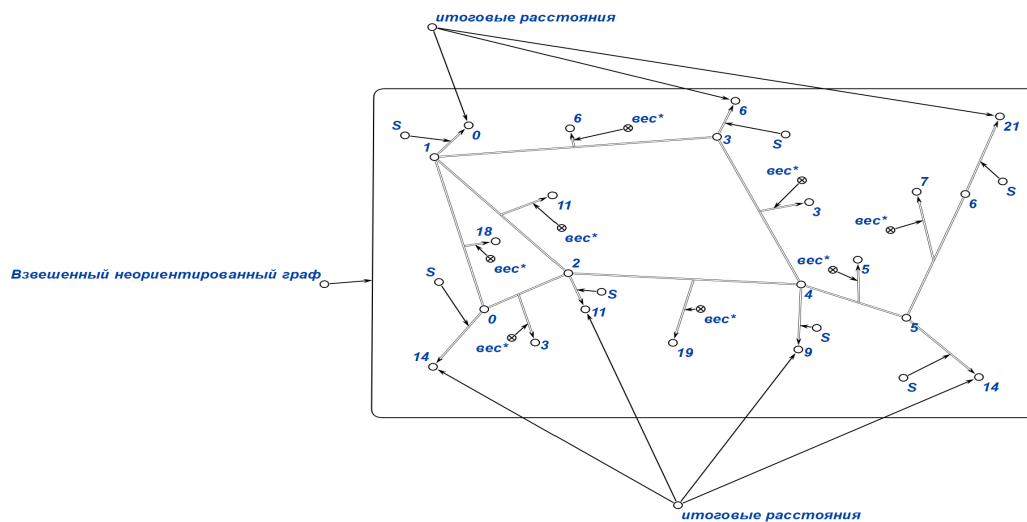


Рис. 4: Итоговые пути:14,0,11,6,9,14,21

5.2 Тест 2

В этом тестовом примере стартовая вершина 4

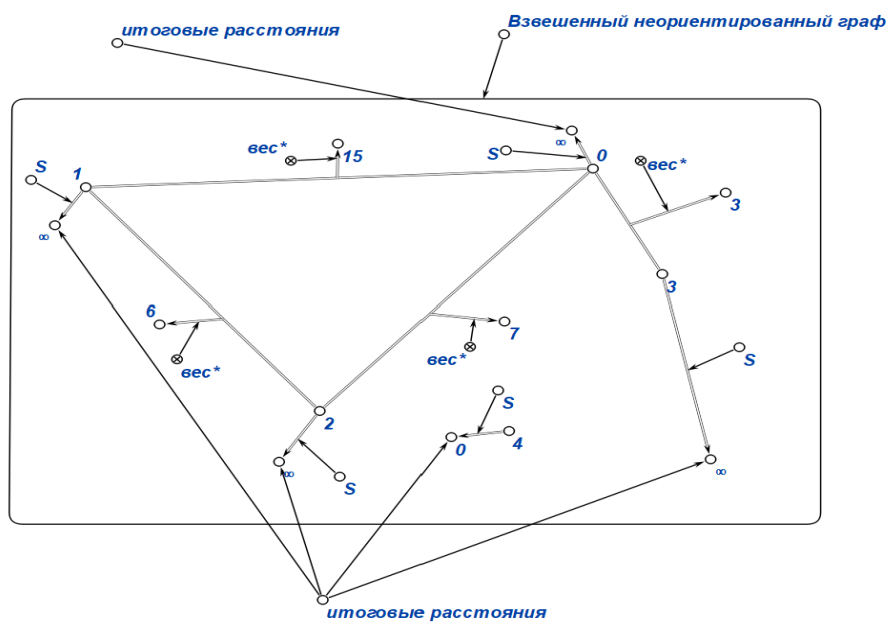


Рис. 5: Итоговые пути: $\infty, \infty, \infty, \infty, 0$

5.3 Тест 3

В этом тестовом примере стартовая вершина **7**

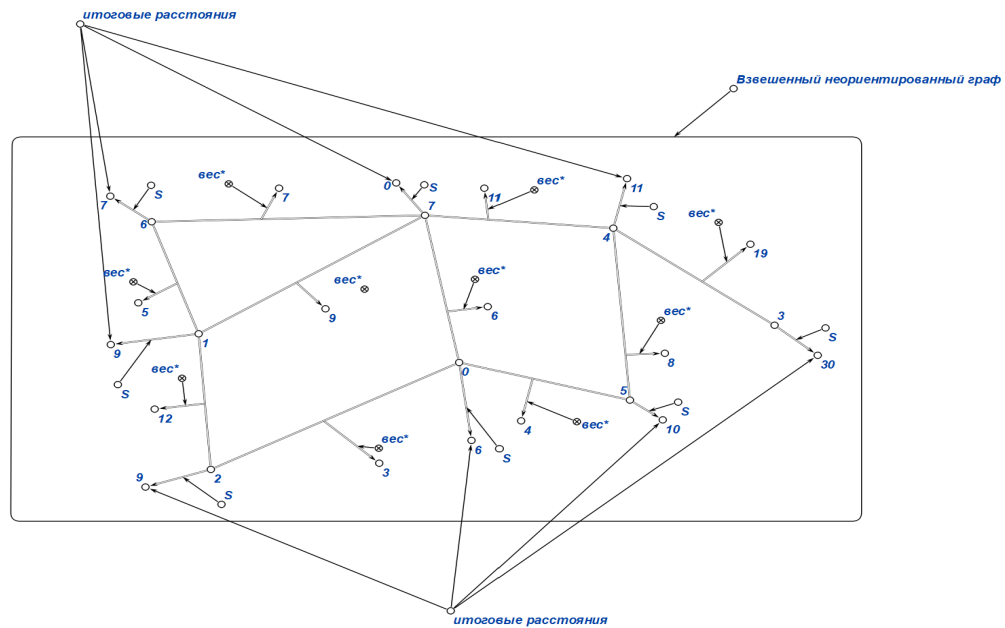


Рис. 6: Итоговые пути: 6,9,9,30,11,10,7,0

5.4 Тест 4

В этом тестовом примере стартовая вершина **0**

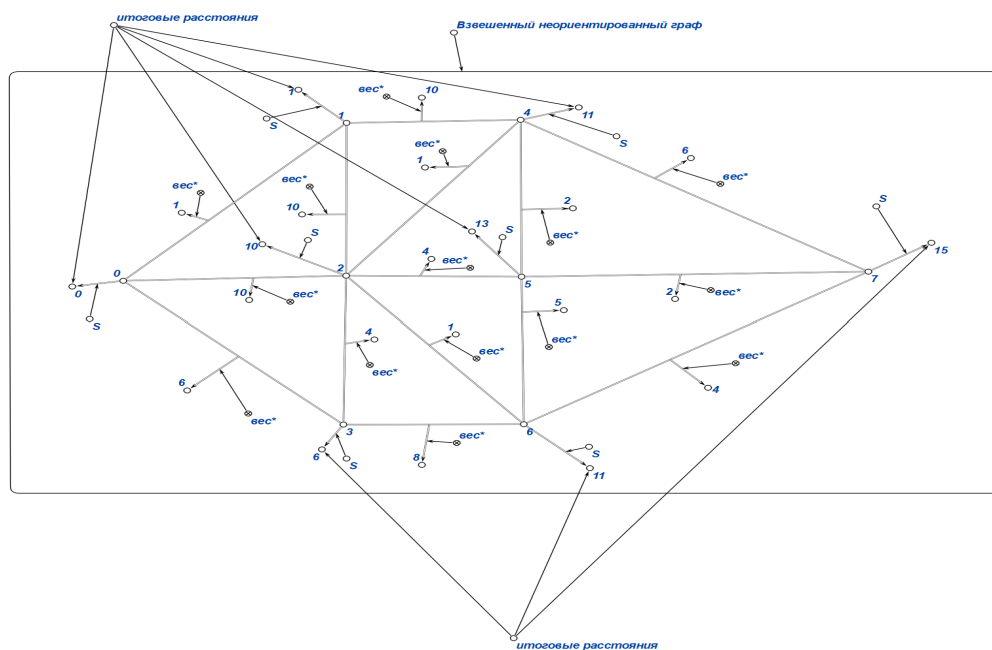


Рис. 7: Итоговые пути: 0,1,10,6,11,13,11,15

5.5 Тест 5

В этом тестовом примере стартовая вершина **3**

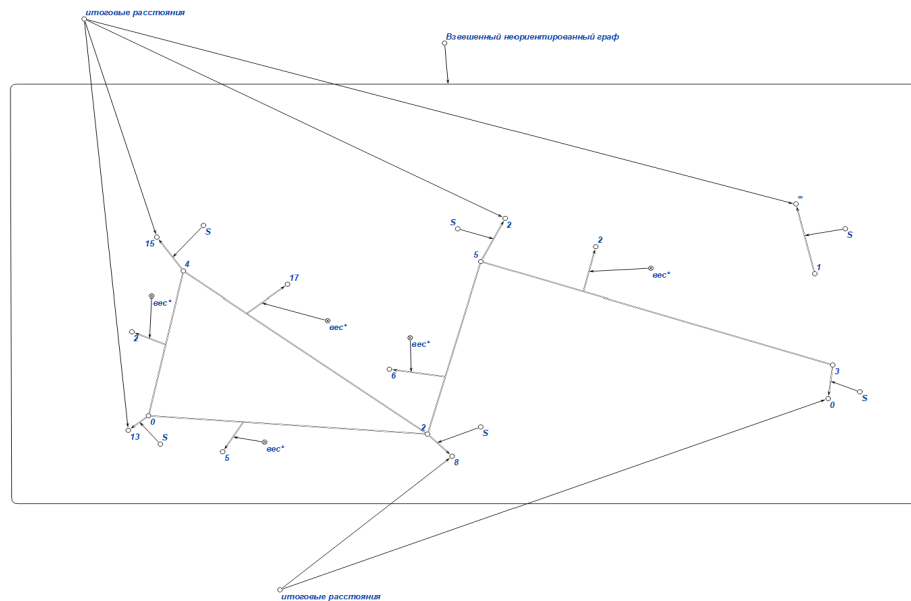


Рис. 8: Итоговые пути: $13, \infty, 8, 0, 15, 2$

6 Алгоритм работы программы

Алгоритм Дейкстры:

- 1) Создание массива путей, который хранит в себе расстояния от стартовой вершины до всех остальных, и изначально заполнен бесконечностями (ещё не достигнута).
- 2) Создание очереди(с приоритетом) для хранения непосещенных вершин и кратчайших расстояний до них.
- 3) Получение стартовой вершины.
- 4) В массиве расстояний на месте стартовой вершины ставим вместо бесконечности 0.
- 5) В очередь помещаем стартовую вершину.
- 6) Если очередь пуста,выполняем пункт 7).
- 6.1) Получаем номер вершины и расстояние до нее из очереди.
- 6.2) Удаляем вершину из очереди.
- 6.3) Если полученное расстояние больше расстояния, которое храниться в массиве путей, то переходим к пункту 6.
- 6.4) Если вершина не имеет соседей(то есть вершины в которые можно попасть за проход одного ребра), то выполняем пункт 6.
- 6.5) Извлекаем вершину в которую идем и расстояние до нее из исходного графа.
- 6.6) Если расстояние вершины, в которую идем, в массиве путей меньше либо равно суммы расстояния в массиве путей от вершины, которую извлекли из очереди(исходная), и длины ребра от исходной вершины, в которую идем, то выполняем пункт 6.4.

7.2 Извлекаем из очереди вершину 0

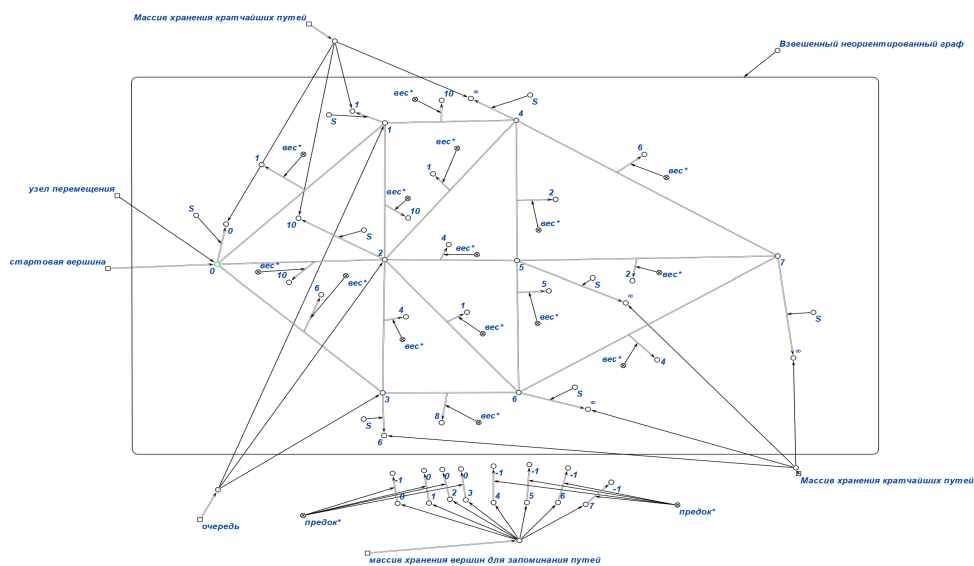


Рис. 10: Итоговые пути: 0, 1, 10, 6, ∞, ∞, ∞, ∞

7.3 Извлекаем из очереди вершину 1

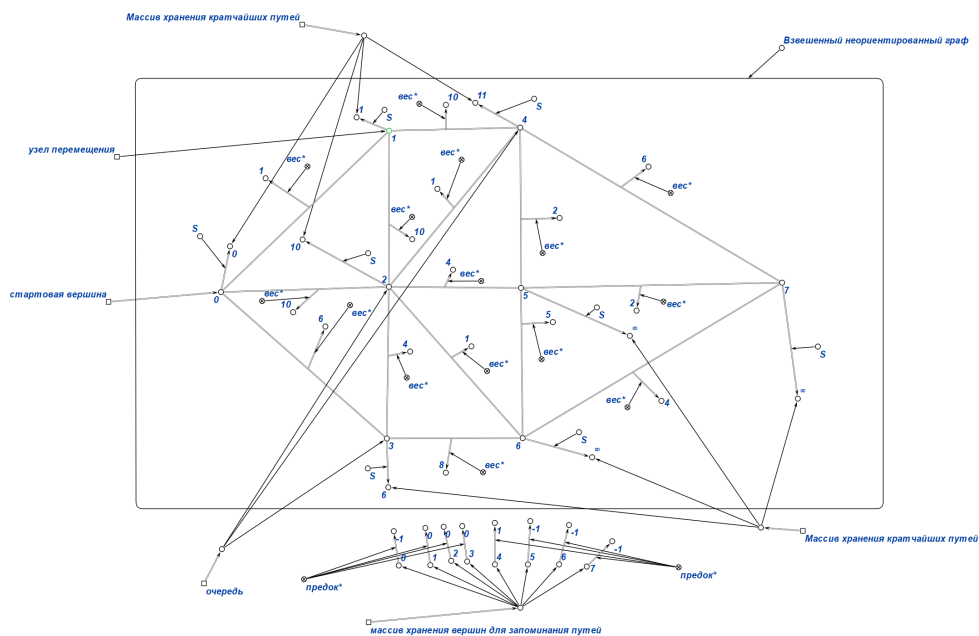


Рис. 11: Итоговые пути: 0, 1, 10, 6, 11, ∞, ∞, ∞

7.4 Извлекаем из очереди вершину 3

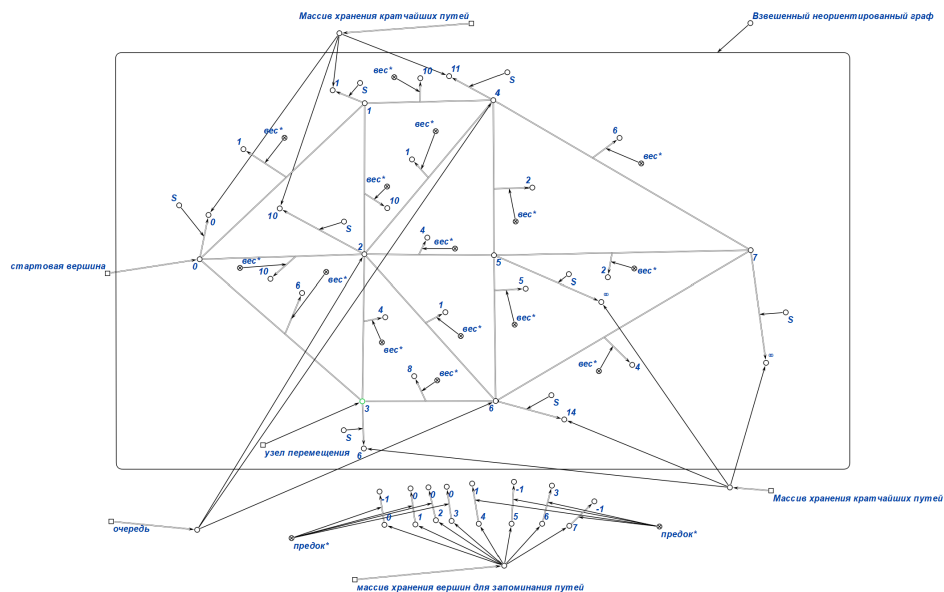


Рис. 12: Итоговые пути:0,1,10,6,11,∞,14,∞

7.5 Извлекаем из очереди вершину 2

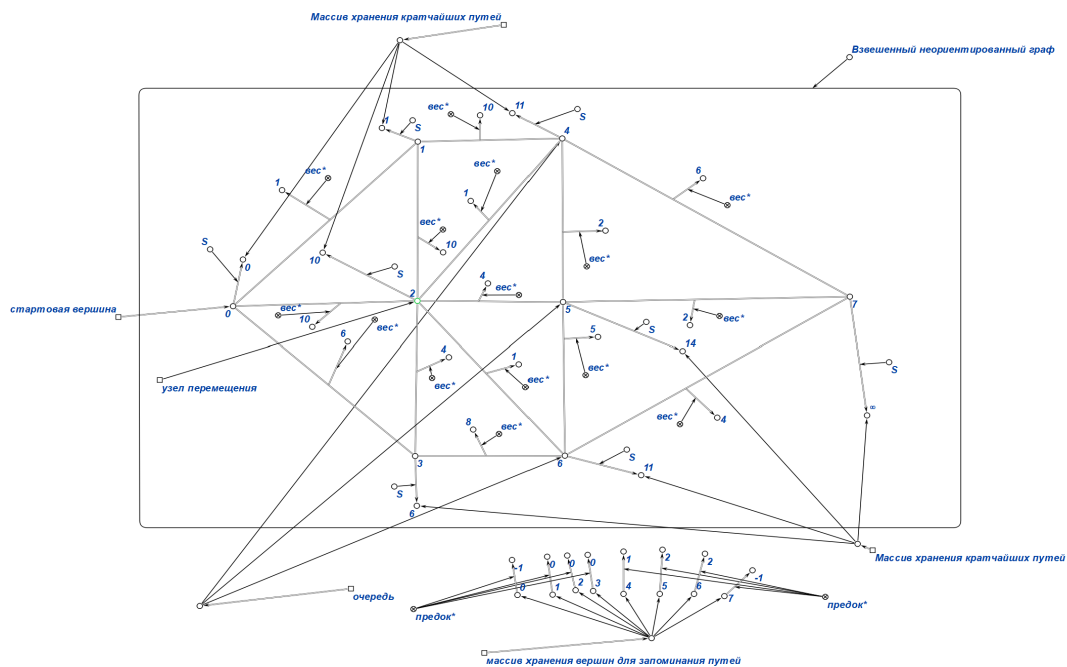


Рис. 13: Итоговые пути:0,1,10,6,11,14,11,∞

7.6 Извлекаем из очереди вершину 4

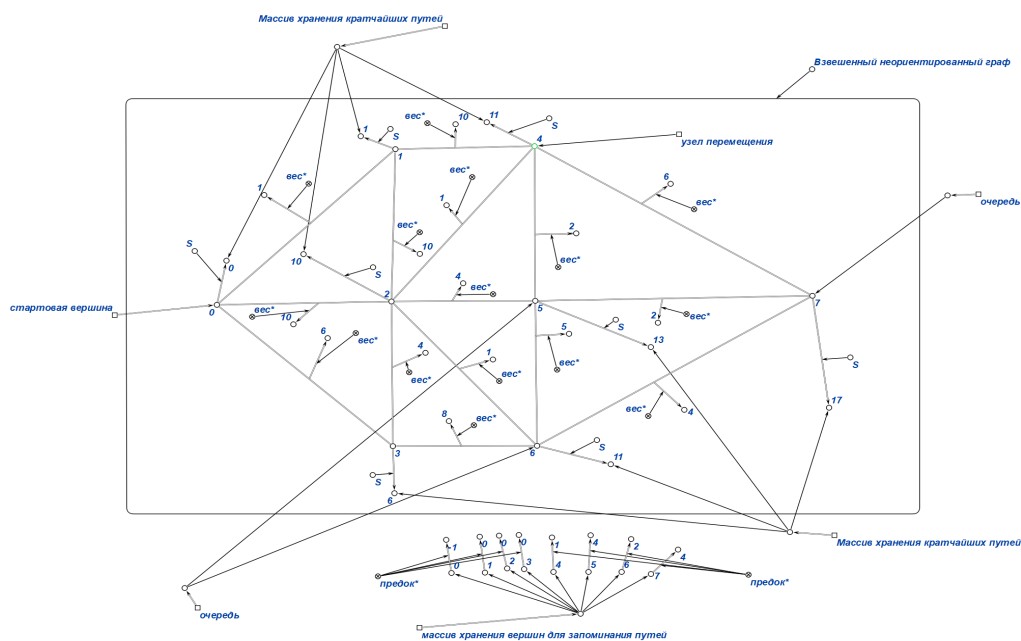


Рис. 14: Итоговые пути: 0,1,10,6,11,13,11,17

7.7 Извлекаем из очереди вершину 6

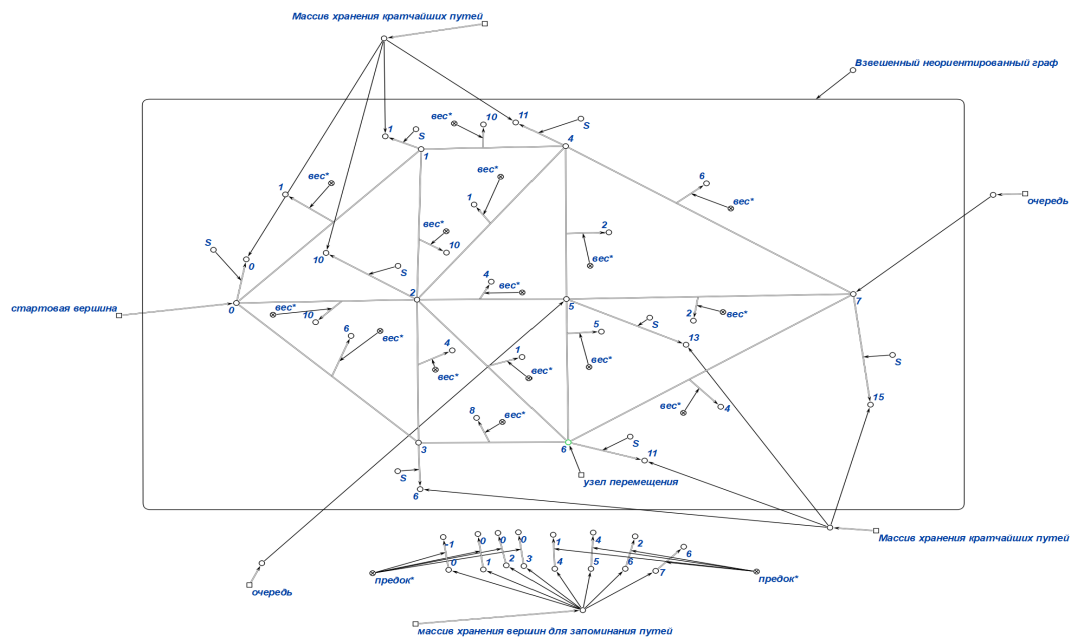


Рис. 15: Итоговые пути:0,1,10,6,11,13,11,15

7.8 Извлекаем из очереди вершину 5

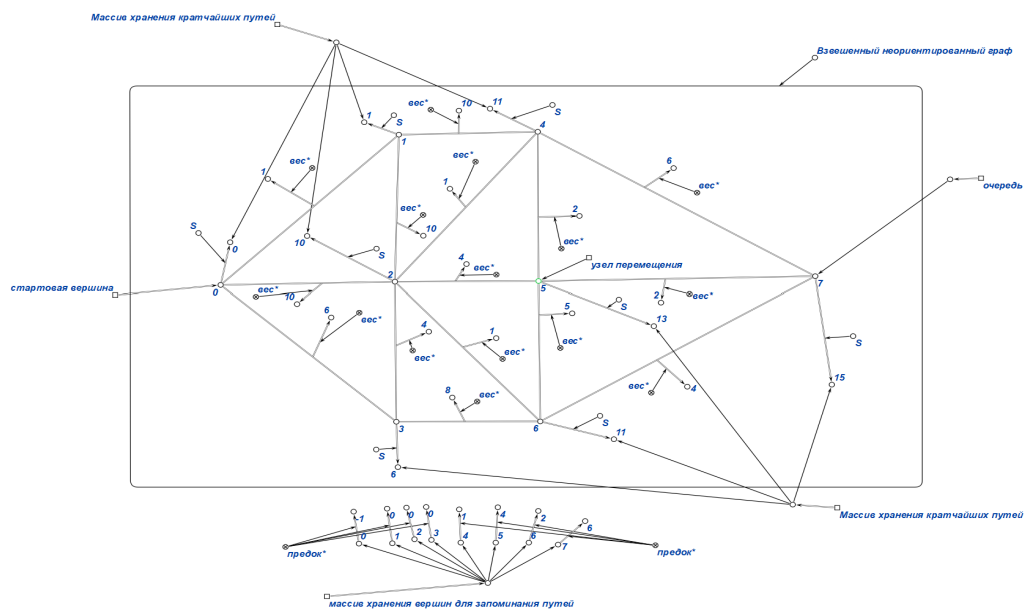


Рис. 16: Итоговые пути:0,1,10,6,11,13,11,15

7.9 Извлекаем из очереди вершину 7

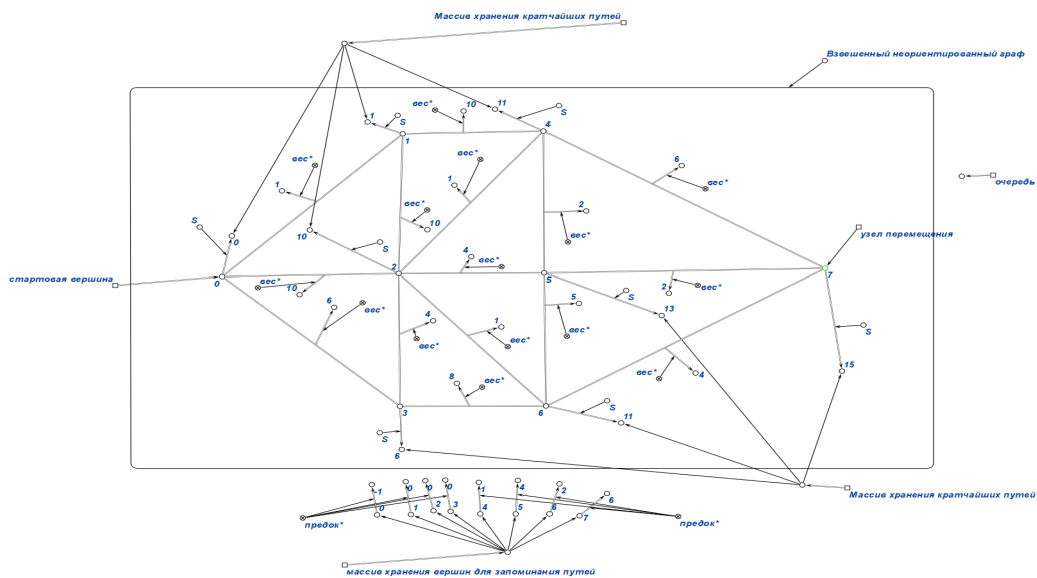


Рис. 17: Итоговые пути:0,1,10,6,11,13,11,15

7.10 Кратчайшие пути

Также алгоритм можно сделать и с восстановлением пути, тогда алгоритм Дейкстры будет выглядеть так:

- 1) Создание массива путей, который хранит в себе расстояния от стартовой вершины до всех остальных, и изначально заполнен бесконечностями (ещё не достигнута).
- 2) Создание очереди(с приоритетом) для хранения непосещенных вершин и кратчайших расстояний до них.
- 3) Создание массива, который хранит в себе последовательность вершин(путь, только записанный в обратном порядке), изначально заполнен **-1**.
- 4) Получение стартовой вершины.
- 5) В массиве расстояний на месте стартовой вершины ставим вместо бесконечности 0.
- 6) В очередь помещаем стартовую вершину.
- 7) Если очередь пуста,выполняем пункт 8.
- 7.1) Получаем номер вершины и расстояние до нее из очереди.
- 7.2) Удаляем вершину из очереди.
- 7.3) Если полученное расстояние больше расстояния, которое храниться в массиве путей, то переходим к пункту 7.
- 7.4) Если вершина не имеет соседей(то есть вершины в которые можно попасть за проход одного ребра), то выполняем пункт 7.
- 7.5) Извлекаем вершину в которую идем и расстояние до нее из исходного графа.
- 7.6) Если расстояние вершины, в которую идем, в массиве путей меньше либо равно суммы расстояния в массиве путей от вершины, которую извлекли из очереди(исходная), и длины ребра от исходной вершины, в которую идем, то выполняем пункт 7.4.
- 7.7) Проводим релаксацию, то есть в массиве путей вершине, в которую идем присваиваем сумму расстояния в массиве путей от вершины, которую извлекли из очереди(исходная), и длины ребра от исходной вершины, в которую идем.
- 7.8) В массиве, который хранит последовательность вершин(путь), на индекс номер,которого равен вершине в которую идем, ставим номер вершины из которой идем.
- 7.9) Добавляем вершину в очередь.
- 7.10) Выполняем пункт 7.4.
- 8) Алгоритм завершается.
- 9) После окончания работы алгоритма используем новый массив для восстановления пути в обратном направлении.(так как изначально записывался в обратном порядке).

Демонстрацию кратчайших путей я продемонстрирую на примере одной вершины(чтобы вывести кратчайшие пути от стартовой до всех остальных, необходимо просто в создать цикл, который делает столько итераций, сколько вершин у графа, и в этом цикле в качестве конечной переменной брать номер итерации). Стартовая вершина - **0**, а конечная - **4**.

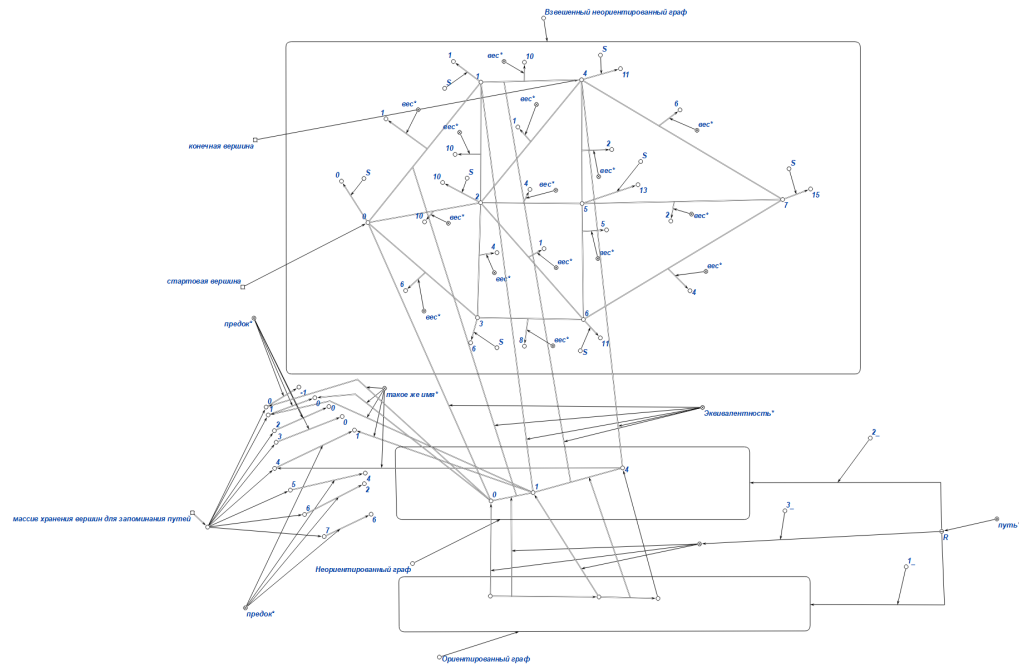


Рис. 18: Итоговые пути: 0->1->4

8 Вывод:

В результате расчетной работы я укрепил знание алгоритма Дейкстры, формализовал его в редакторе **КВЕ**, привел тестовые примеры и проиллюстрировал шаг за шагом работу алгоритма.

9 Используемые источники:

- Сайт "Олимпиадное программирование в Бресте и Беларуси" [Электронный ресурс] - режим доступа: <https://brestprog.by/topics/dijkstra/>
[Алгоритм Дейкстры](#)
- Сайт Свободная энциклопедия "Википедия" [Электронный ресурс] - режим доступа: https://ru.m.wikipedia.org/wiki/Базовая_информация_по_графам
- Skillfactory Blog. Алгоритм Дейкстры [Электронный ресурс]. Режим доступа: <https://blog.skillfactory.ru/glossary/algorithm-dejkstry/>
[Визуализация алгоритма Дейкстры](#)